3.3 Overriding and polymorphism

An override provides a subclass implementation of an inherited method. Polymorphism lets calling code use a common type while each object's implementation determines the result.

Replace an inherited method

Run this complete example in `3-3-overriding.php`:

```php
<?php
declare(strict_types=1);

class Food
{
    public function __construct(protected string $name)
    {
    }

    public function __toString(): string
    {
        return 'Food: ' . $this->name;
    }
}

class Dessert extends Food
{
    public function __toString(): string
    {
        return 'Dessert: ' . $this->name;
    }
}

function describe(Food $food): string
{
    return (string) $food;
}

foreach ([new Food('Apple'), new Dessert('Berry tart')] as $food) {
    echo describe($food), PHP_EOL;
}
```

```
Food: Apple
Dessert: Berry tart
```

Trace: The parameter says Food, but the second object is still a Dessert. Converting it to a string invokes Dessert's method. No type switch is needed. Dessert inherited Food's constructor, and overriding

`__toString()` did not replace that constructor or create another object.

The parent version of `__toString()` did not automatically run first. The next tutorials add a check for override intent and show how to deliberately reuse the parent's body.

Keep the parent's promises

The **Liskov substitution principle** says that a child object should work wherever a parent object is expected without violating the expected behavior. A method can have legal syntax and still break a promise—for example, a child method named `getLabel()` that unexpectedly deletes a record.

Chapter 19's sidebar covers these compatible signature variations. The entries below are declarations to compare, not a single runnable class:

Parent declaration	Compatible child declaration	Why old callers still work
`check(int $age): bool`	`check(int $age = 18): bool`	Required argument becomes optional
`check(int $age): bool`	`check(int $age, bool $strict = false): bool`	Added argument is optional
`choose(): Food`	`choose(): Dessert`	Result is more specific: covariance
`accept(Dessert $food): void`	`accept(Food $food): void`	Input is more general: contravariance

Keep identical signatures while learning unless a variation has a clear purpose. Do not add a required argument, narrow an accepted parameter type, widen the return type, or reduce public visibility to protected. Preserve parameter names too: callers can use named arguments even though PHP allows names to differ. PHP checks signature compatibility; it cannot prove the whole behavioral contract. See the PHP inheritance reference.

Constructors are generally exempt from these signature compatibility rules: a child can require additional construction data. That does not make initialization optional; tutorial 3.5 explains the parent's constructor.